Movie Recommendation System: User-Based Collaborative Filtering

Group ID: 32

Lujain Alsubhi | 25117577

Mana Al Manssor |

Abstract

This report addresses the challenge of content discovery in the era of overwhelming media availability across streaming platforms. As users face difficulty finding suitable content on movie streaming services, we present a recommendation engine based on collaborative filtering techniques. Our system implements multiple approaches—Singular Value Decomposition, K-Nearest Neighbors, Approximate Nearest Neighbors, and a hybrid method—to predict user preferences based on the viewing patterns of similar users. By analyzing user-item matrices and identifying rating patterns, the recommendation engine aims to reduce browsing time and improve content discovery, ultimately enhancing the user experience by connecting viewers with movies that match their individual tastes.

Table of contents

Tables

Figures

I. Recommendation System

1. Problem statement

Content availability across streaming platforms, online retailers, and media services has reached unprecedented heights. In the context of movie streaming services specifically, users struggle to find content that fits their preferences, and without helpful recommendations, they can spend excessive time browsing only to settle on suboptimal matches for their expectations. Recommendation systems help address this issue by analyzing user preferences and behavior to suggest content that aligns with individual tastes. This project focuses on building a recommendation engine that predicts movie ratings using collaborative filtering, helping users find content they're more likely to enjoy.

2. Proposed Solution

To address the problem, we implemented a **user-based collaborative filtering algorithm** using multiple approaches:

- Singular Value Decomposition
- K-Nearest Neighbors (KNN)
- Approximate Nearest Neighbors (Annoy)
- Hybrid approach

The system uses the ratings of similar users to predict the preferences of a given user. By constructing a user-item matrix and profiling users and movies, the system identifies similar patterns and neighbors to generate predictions based on their ratings.

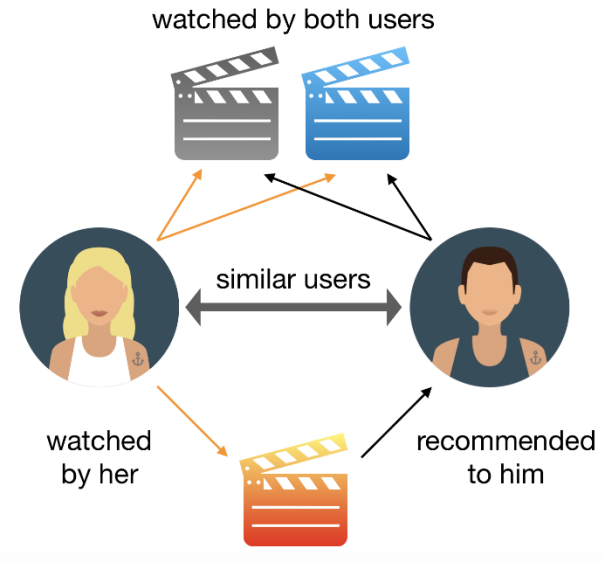

Figure 1: User-based recommendation (Le, 2019)

II. Theoretical Foundation

1. Collaborative Filtering

The fundamental concept underpinning collaborative filtering is to use user ratings of certain documents to suggest those same publications to other users without having to analyze the content of the documents. Faruk, K. (2022).

Advantages

Does not require metadata about items

Unlike content-based systems, collaborative filtering doesn't require detailed metadata about items. It works purely based on user interactions, making it useful for recommending complex items like movies where preferences are subjective. Xu & Han (2021)

Can recommend items not previously interacted with by the user

The system can recommend items the user hasn't seen before as long as similar users have interacted with them. This helps users discover content they might not have found on their own.

Disadvantages

The *cold start* problem:

It's difficult to recommend relevant content when there's little to no data about new users or new items. Kumar et al. (2021).

Data sparsity

Most users only interact with a small portion of available items, making it harder to draw meaningful connections between users and preferences. Sadhasivam et al. (2021)

Scalability

As user and item databases grow, the calculations required become increasingly resource-intensive. Processing millions of users and items demands significant computational power, especially for real-time recommendations.

2. Single Values Decomposition (SVD)

The singular value decomposition of a matrix A is the factorization of A into the product of three matrices $A = UDV^T$, where the columns of U and V are orthonormal and the matrix D is diagonal with positive real entries. (Carnegie Mellon University)

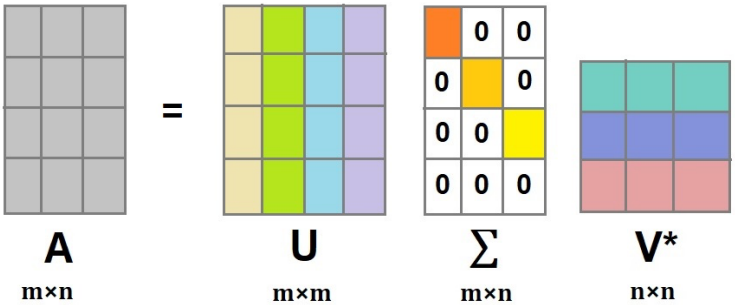

Figure 2: SVD decomposition (AskPython, 2020)

The SVD is especially useful for problems with high dimensionalities, which is one of the main difficulties with collaborative filtering systems. From the singular value decomposition of A, we can get the matrix B of rank k which best approximates A. Also, singular value decomposition is defined for all matrices (rectangular or square) unlike the more commonly used spectral decomposition in Linear Algebra. Skotis & Livas (2024)

By reducing the dimensionality of the user-item interaction matrix, SVD uncovers underlying patterns and relationships between users and items. The latent factors capture hidden characteristics, such as user preferences or item attributes that contribute to user-item interactions. Faruk, K. (2022)

Tools used

Scikit-learn's TruncatedSVD:

This transformer performs linear dimensionality reduction by means of truncated singular value decomposition (SVD). Contrary to PCA, this estimator does not center the data before computing the singular value decomposition. This means it can work with sparse matrices efficiently. Sharma & Shakya (2024)

Advantages

Optimal For high dimensional problems

Reduces the number of features while minimizing loss of information.

Adapted to sparse matrices:

Most user-item matrices for recommendation engines are sparse matrices, which is expected as most users haven't rated most items.

Using a dense matrix would be more than suboptimal, as it's too computationally expensive. For instance, it simply crashed the standard Google Colab session.

Disadvantages

Decrease in accuracy

As it is with most things, using SVD comes with a compromise of losing some information and therefore a reduction in prediction quality. A balance should be struck between computational feasibility and prediction quality.

3. K-Nearest Neighbors (KNN)

KNN is a non-parametric algorithm that classifies or predicts a value based on the 'k' closest training examples in the feature space.

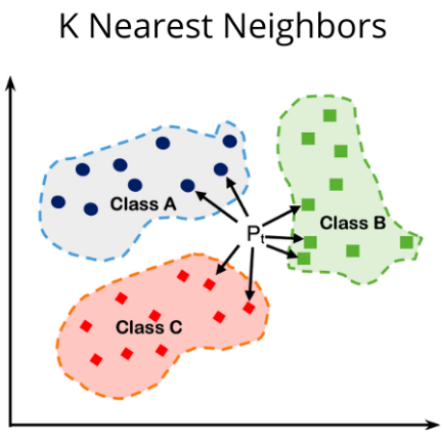

Figure 3: KNN Algorithm working visualization (Sachinsoni, 2023)

Advantages

Simple to understand and implement.

Effective for small- to medium-sized datasets.

Disadvantages

Dimensionality curse

The "dimensionality curse" makes similar calculations increasingly unreliable as item counts grow into the thousands, significantly hampering effectiveness.

Scalability

Computational demands present another barrier—KNN requires storing all training data and calculating distances across all examples, creating prohibitive resource needs for large user bases. Verma & Diwan (2023).

Implementation requires optimization techniques like dimensionality reduction and approximate neighbor methods to remain viable at scale.

Parameter-dependent

KNN's performance heavily depends on the fixed 'k' value, which represents the number of neighbors considered for making recommendations; a small 'k' might lead to noisy recommendations, while a large 'k' can introduce undue influence from distant neighbors.Faruk, K. (2022)

Tools used

Scikit-learn's NearestNeighbors

Cosine similarity

Measures the cosine of the angle between two non-zero vectors, often used to determine how similar two users or items are in recommendation systems.

Annoy

For approximate nearest neighbor search.

III. Preparing the data

1. Data Loading and Preprocessing

The preprocessing phase was critical for making the recommendation system computationally feasible given the large volume of data. We implemented several optimization techniques:

Memory Optimization

- Original data consumption: The raw ratings dataset required significant memory (610 Mega Bytes).

Tableau 1: Raw "ratings" dataframe info()

Column	Dtype
userId	int64
movieId	int64
rating	float64
timestamp	int64

- Data type optimization: Fields were converted to more memory-efficient types:
  - userId to uint32
  - movieId to uint32
  - rating to float32
- Timestamp removal: The timestamp column was dropped as it wasn't needed for our collaborative filtering approach.
- After optimization, memory usage dropped to approximately 229 Mega Bytes (37.50%).

Tableau 2: Optimized 'ratings" dataframe info()

Column	Dtype
userId	uint32
movieId	uint32
rating	float32

Data Cleaning

- Duplicate entries were removed to prevent bias.
- Rows with missing values were dropped to ensure data integrity.

Sparsity Reduction

- A threshold of 50 ratings was established for both users and movies.
  - Users with fewer than 50 ratings were filtered out to remove casual users with insufficient data.
  - Movies with fewer than 50 ratings were filtered out to eliminate obscure titles with minimal feedback.
- This filtering process significantly reduced matrix sparsity and improved computational efficiency.

Index resetting

- After filtering, indices were reset to reclaim memory and ensure contiguous data access.
- This step improved memory locality, leading to better cache utilization during computation.

The resulting preprocessed dataset retained only the most informative data points, striking a balance between representativeness and computational manageability.

2. Data Preparation and Partitioning

Partitioning

The dataset was systematically divided into training (80%) and test (20%) sets to enable robust model evaluation.

Global variables' definition

```
USER_CATEGORIES = train_data['userId'].astype('category').cat.categories
MOVIE_CATEGORIES = train_data['movieId'].astype('category').cat.categories
GLOBAL_MEAN = train_data['rating'].mean()
```

'user-movie' matrix

The training data is transformed into a sparse user-movie matrix where rows represent unique users and columns represent individual movies. To improve model performance, we normalized the ratings by subtracting the global mean rating from each value in the matrix. This normalization step removes general rating bias, allowing the models to focus on relative preferences. During prediction, this global mean is subsequently added back to generate final rating estimates.

Mapping of users & movies

```
user_ids = train_data['userId'].astype('category')
movie_ids = train_data['movieId'].astype('category')

# Optionally store mappings
user_mapping = dict(enumerate(user_ids.cat.categories))
movie_mapping = dict(enumerate(movie_ids.cat.categories))

user_reverse_mapping = {v: k for k, v in user_mapping.items()}  # real userId → int
movie_reverse_mapping = {v: k for k, v in movie_mapping.items()}# real movieId → int
```

An essential component of our implementation includes bidirectional mapping systems between the sparse matrix indices and the original user and movie IDs. These mapping dictionaries serve multiple critical functions:

- Facilitating efficient lookup between matrix positions and meaningful identifiers
- Enabling seamless translation between internal model representations and user-interpretable recommendations

- Supporting consistent cross-referencing across different stages of the recommendation pipeline
- Preserving original ID integrity while optimizing computational efficiency

IV. Model Implementation and Prediction Methodology

1. K Nearest Neighbors (KNN)

1.1. Implementation overview

This straightforward approach involves two primary steps:

Model initialization

- Distance metric: Cosine similarity
- Algorithm: Brute force computation

Model fitting

The model is trained on the user-movie sparse matrix, providing a comprehensive similarity framework. For prediction, when evaluating a specific user-movie pair, the algorithm:

1. Identifies the k users most similar to the target user
2. Retrieves these users' ratings for the specific movie
3. Calculates a weighted average of these ratings based on user similarity
4. Returns this average as the predicted rating

1.2. Implementation details

Sparse matrix transformer

We define a function to build the sparse matrix

```
def build_sparse_matrix(df):
    user_codes = pd.Categorical(df['userId'], categories=USER_CATEGORIES).codes
    movie_codes = pd.Categorical(df['movieId'], categories=MOVIE_CATEGORIES).codes

    return csr_matrix(
        (df['rating'] - GLOBAL_MEAN, (user_codes, movie_codes)),
        shape=(len(USER_CATEGORIES), len(MOVIE_CATEGORIES))
    )
```

Based on this function, we create a sparse matrix transformer using the 'FunctionTransformer' class.

```
sparse_transformer = FunctionTransformer(build_sparse_matrix, validate=False)
```

Predictor definition

Define the KNN predictor with the set parameters:

```
knn = NearestNeighbors(
        metric='cosine',
```

```
        algorithm='brute'
    )
```

Pipeline

The sequence of data transformers and the predictor can then be organized into a single object with the help of the 'Pipeline' class.

```
pipeline = Pipeline([
    ('sparse_matrix', sparse_transformer),
    ('KNN', knn)
])
```

Which can be fit into with the training data with a single line

```
pipeline.fit(train_data)
```

Rating predictions

We can predict a movie's rating by a user through a custom function, which performs the following tasks:

- Neighbor discovery: Finds the k most similar users to the target user using the KNN model
- Rating collection: Gathers ratings from these neighbor users for the specific movie in question
- Weighted prediction: Calculates a predicted rating by taking a weighted average of the neighbors' ratings, where closer neighbors have more influence
- Fallback handling: Returns a global mean rating when there's insufficient data

```
def predict_knn_rating(pipeline, train_data, user_id, movie_id, k=5):
    user_movie_matrix = pipeline.named_steps['sparse_matrix'].transform(train_data)
    knn_model = pipeline.named_steps['KNN']

    if user_id not in user_reverse_mapping or movie_id not in movie_reverse_mapping:
        return GLOBAL_MEAN  # Return global mean for unknown users/items

    user_idx = user_reverse_mapping[user_id]
    movie_idx = movie_reverse_mapping[movie_id]

    user_vector = user_movie_matrix.getrow(user_idx)

    # Find nearest neighbors (including the user themselves)
    distances, indices = knn_model.kneighbors(user_vector, n_neighbors=k+1)

    # Exclude the user themselves (first neighbor)
    neighbors = indices.flatten()[1:]
    neighbor_distances = distances.flatten()[1:]

    ratings = []
```

```
    weights = []

    for neighbor_idx, dist in zip(neighbors, neighbor_distances):
        # Get the neighbor's rating for the target movie
        rating = user_movie_matrix[neighbor_idx, movie_idx]
        if rating != 0:  # Only consider actual ratings (non-zero in sparse matrix)
            ratings.append(rating)
            weights.append(1 - dist)  # Convert distance to similarity

    if not ratings:
        return GLOBAL_MEAN

    # Calculate weighted average of ratings and add back global mean
    weighted_avg = np.average(ratings, weights=weights)
    return weighted_avg + GLOBAL_MEAN
```

Recommendation

A second function is defined to get the top ‘k’ movie recommendations for a movie:

- Batch Processing Setup: Processes multiple users at once and identifies all available movies and each user’s already-rated movies
- Neighbor-Based Rating Prediction: Fore each user, find ‘k’ similar neighbors and computes predicted ratings for all movies using weighted neighbor preferences
- Filtering: Removes movies the user has already rated to focus only on new recommendations
- Top-K selection: Ranks unrated movies by predicted ratings and selects the top ‘k’ highest-rated movies as recommendations for each user

```
def get_batch_recommendations(pipeline, train_data, user_ids, k=5, top_n=10):

    user_movie_matrix = pipeline.named_steps['sparse_matrix'].transform(train_data)
    knn_model = pipeline.named_steps['KNN']
    all_movie_ids = movie_ids.cat.categories
    user_rated_movies = train_data.groupby('userId')['movieId'].apply(set)

    recommendations = {}

    for user_id in user_ids:
        if user_id not in user_reverse_mapping:
            recommendations[user_id] = np.array([])
            continue

        user_idx = user_reverse_mapping[user_id]
        user_vector = user_movie_matrix.getrow(user_idx)
```

```
        # Find neighbors
                distances,   neighbor_indices   =   knn_model.kneighbors(user_vector,
n_neighbors=k+1)
        neighbor_indices = neighbor_indices.flatten()[1:]
        similarity_weights = 1 - distances.flatten()[1:]

        # Compute weighted ratings
                                                        weighted_ratings            =
user_movie_matrix[neighbor_indices].multiply(similarity_weights[:,
np.newaxis]).sum(axis=0)
        weighted_ratings = np.array(weighted_ratings).flatten() + GLOBAL_MEAN

        # Filter out already rated movies
        rated = user_rated_movies.get(user_id, set())
        unrated_mask = [movie_id not in rated for movie_id in all_movie_ids]
        unrated_ratings = weighted_ratings[unrated_mask]
        unrated_movies = all_movie_ids[unrated_mask]

        # Get top recommendations
        top_indices = np.argsort(unrated_ratings)[-top_n:][::-1]
        recommendations[user_id] = unrated_movies[top_indices]

    return recommendations
```

2. Approximate Nearest Neighbors (Annoy)

The Annoy algorithm provides an efficiency-optimized alternative to traditional KNN, trading some prediction accuracy for substantially improved computational performance.

2.1. Implementation overview

Key Configuration Parameters:

- Number of trees: 20
- Distance metric: Angular (equivalent to cosine distance)

The resulting Annoy index enables rapid approximate neighbor retrieval while maintaining reasonable prediction quality.

2.2. Implementation details

Sparse matrix

Since Annoy requires dense vector representations (incompatible with our large sparse matrix), we implemented a row-by-row conversion process to avoid memory constraints.

```
def fit_annoy_model(sparse_matrix, num_trees=20):
    # Annoy expects dense vectors, so we need to convert the sparse matrix to dense
```

```
    # Here we convert the matrix row by row to avoid holding the full dense matrix in
memory.
    num_features = sparse_matrix.shape[1]
    annoy_index = AnnoyIndex(num_features, 'angular')  # 'angular' is commonly used
for cosine distance

    for idx in range(sparse_matrix.shape[0]):
        # Convert the sparse row to a dense vector
        vector = sparse_matrix[idx].toarray().flatten()
        annoy_index.add_item(idx, vector)

    annoy_index.build(num_trees)  # Build the index with the specified number of trees
    return annoy_index
```

3. Singular Vector Decomposition (SVD)

3.1. Implementation overview

We leveraged scikit-learn's implementation of SVD with the following configuration:

Model Parameters:

- Latent dimensions (n_components): Varied between 50-500
- Algorithm: Randomized approach
- Iterations: 10
- Oversamples: 20
- Power iteration normalizer: Auto
- Random state: 42
- Tolerance: 10^-5

The trained model produces two key matrices: a left singular matrix representing users and a right singular matrix representing movies. The matrix product of these reduced-dimension representations generates our prediction matrix.

3.2. Implementation details

Sparse matrix transformer

Same as before, we instantiate a transformer object for the sparse matrix in the same manner.

Predictor definition

Define the SVD predictor with the set parameters:

```
algorithm = 'randomized'
n_iter = 10
n_oversamples = 20
power_iteration_normalizer = 'auto'
random_state = 42
tol = 1e-5
```

```
svd = TruncatedSVD(
        n_components=n,
        algorithm='randomized',
        n_iter=n_iter,
        n_oversamples=n_oversamples,
        power_iteration_normalizer=power_iteration_normalizer,
        random_state=random_state,
        tol=tol
    )
```

Pipeline

Instantiate and fit the pipeline with all the transformations and predictors:

```
pipeline = Pipeline([
    ('sparse_matrix', sparse_transformer),
    ('SVD', svd)
])
pipeline.fit(train_data)
```

Rating predictions

- SVD-Based Rating Prediction: Uses Singular Value Decomposition to create user and movie embeddings, then computes predicted ratings through matrix multiplication of these low-dimensional representations.
- Score Processing: Adjusts predictions by adding the global mean and clamps values to valid rating bounds (0.5-5.0), with fallback to global mean for unknown users/movies.

```
def predict_rating(pipeline, train_data, row):
    user_movie_matrix = pipeline.named_steps['sparse_matrix'].transform(train_data)
    svd_model = pipeline.named_steps['SVD']

    user_embeddings = svd_model.transform(user_movie_matrix)  # Transform the matrix,
not the raw data
    movie_embeddings = svd_model.svd.components_.T  # Access svd through the hybrid
model

    predicted_matrix = user_embeddings @ movie_embeddings.T

    user = row['userId']
    movie = row['movieId']

    if user in user_reverse_mapping and movie in movie_reverse_mapping:
        u = user_reverse_mapping[user]
        i = movie_reverse_mapping[movie]

        svd_pred = predicted_matrix[u, i]
```

```
        prediction = svd_pred + GLOBAL_MEAN
        return np.clip(prediction, 0.5, 5.0)
    else:
        return GLOBAL_MEAN  # Better than returning nan
```

Recommendation

- Relevance Analysis: Identify which test movies are considered “relevant” for each user based on a rating threshold, creating ground truth for evaluation
- SVD-Based Score Generation: Uses SVD to create user and movie embeddings, then computes predicted rating scores for all user-movie pairs through matrix multiplication
- Recommendation Filtering: Optionally excludes movies users have already seen in training data to focus on novel recommendations
- Top-K Selection: Ranks movies by predicted scores and returns the k highest scoring recommendations for each user, along with their relevant test items for performance evaluation

```
def get_batch_recommendations(pipeline, train_data, test_data=None, k=50,
                              relevance_threshold=4.0, exclude_seen=True):

    # 1. Prepare relevance data if test_data provided
    user_relevant_test_items = defaultdict(set)
    if test_data is not None:
        test_data['relevant'] = test_data['rating'] >= relevance_threshold
        for _, row in test_data.iterrows():
            if row['relevant']:
                user_relevant_test_items[row['userId']].add(row['movieId'])

    # 2. Get embeddings and predictions
    user_movie_matrix = pipeline.named_steps['sparse_matrix'].transform(train_data)
    svd_model = pipeline.named_steps['SVD']

    user_embeddings = svd_model.transform(user_movie_matrix)
    movie_embeddings = svd_model.svd.components_.T
    predicted_matrix = user_embeddings @ movie_embeddings.T

    # 3. Generate recommendations
    recommendations = {}
          user_ids    =    user_relevant_test_items.keys()    if    test_data    else
train_data['userId'].unique()

    for user_id in user_ids:
        if user_id not in user_reverse_mapping:
            continue  # Skip users not in training
```

```
        user_idx = user_reverse_mapping[user_id]
        movie_scores = list(enumerate(predicted_matrix[user_idx]))

        # Filter out seen movies if requested
        if exclude_seen:
            seen_movies = set(train_data[train_data['userId'] == user_id]['movieId'])
            movie_scores = [(i, score) for i, score in movie_scores
                            if movie_mapping[i] not in seen_movies]

        # Get top-k recommendations
        top_k = sorted(movie_scores, key=lambda x: x[1], reverse=True)[:k]
        top_k_movie_ids = [movie_mapping[i] for i, _ in top_k]
        recommendations[user_id] = top_k_movie_ids

    return recommendations, user_relevant_test_items
```

4. Hybrid approach (SVD + KNN)

4.1. Implementation overview

Our hybrid methodology combines the strengths of both SVD and KNN approaches:

1. SVD provides dimensional reduction and generates initial rating predictions
2. KNN operates on the SVD-derived movie embeddings rather than raw data, significantly improving computational efficiency
3. Final rating predictions are calculated as a weighted combination of both models' outputs

This approach leverages the dimensional efficiency of SVD while preserving the neighbor-based recommendation logic of KNN.

4.2. Implementation details

Sparse matrix transformer

Once again, the sparse matrix transformer is defined the same way as all other instances

Predictor definition

- This time around, there isn't a pre-built class for an SVD-KNN predictor the way we envision implementing it. Therefore, following the scikit-learn best practices, we define a custom class with the following:
  Combines SVD for both dimensionality reduction and predictions, with KNN for similarity-based recommendations.
- Fits SVD on the user-movie matrix to learn latent factors, then combines its predictions with KNN's on the resulting embeddings, enabling efficient neighbor searches.

```
class SVDToKNNTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, n_components=50, n_neighbors=10, random_state=None):
```

```
        self.n_components = n_components
        self.n_neighbors = n_neighbors
        self.random_state = random_state
        self.svd = TruncatedSVD(
            n_components=n_components,
            algorithm='randomized',
            n_iter=10,
            n_oversamples=20,
            power_iteration_normalizer='auto',
            random_state=random_state,
            tol=1e-5
        )
        self.knn = NearestNeighbors(
            n_neighbors=n_neighbors,
            metric='cosine',
            algorithm='brute'
        )

    def fit(self, X, y=None):
        self.svd.fit(X)
        movie_embeddings = self.svd.components_.T
        self.knn.fit(movie_embeddings)
        return self

    def transform(self, X):
        return self.svd.transform(X)

    def kneighbors(self, X=None, n_neighbors=None, return_distance=True):
        if X is None:
            X = self.svd.components_.T
        return self.knn.kneighbors(X, n_neighbors, return_distance)

svd_knn = SVDToKNNTransformer(
        n_components=50,
        n_neighbors=10,
        random_state=42
    )
```

Pipeline

Instantiate and fit the pipeline with all the transformations and predictors:

```
pipeline = Pipeline([
    ('sparse_matrix', sparse_transformer),
    ('svd_knn', svd_knn)
])
```

```
pipeline.fit(train_data)
```

Rating predictions

- Input Validation: Checks if the user and movie exist in the training data mappings, returning global mean for unknown entities.
- SVD Prediction: Computes baseline rating prediction using matrix factorization by taking the dot product of user and movie embeddings.
- KNN Enhancement: Find similar movies using nearest neighbors and averages ratings from those neighbors to create a collaborative filtering component.
- Hybrid combination: Blends the SVD and KNN predictions (averaging them when both are available), add the global mean offset, and clips the final rating to valid bounds (0.5-5.0)

```
def predict_rating(pipeline, train_data, userId, movieId):
    """Predict rating for a single user-movie pair using SVD+KNN hybrid approach"""

    # Check if user and movie exist in our mappings
    if userId not in user_reverse_mapping or movieId not in movie_reverse_mapping:
        return np.clip(GLOBAL_MEAN, 0.5, 5.0)  # Fallback to global mean

    # Get indices
    u = user_reverse_mapping[userId]
    i = movie_reverse_mapping[movieId]

    # SVD prediction (centered)
    svd_pred = USER_EMBEDDINGS[u] @ MOVIE_EMBEDDINGS[i]

    # KNN-enhanced prediction
    try:
        # Get nearest neighbors
        _, neighbor_indices = HYBRID_MODEL.kneighbors([MOVIE_EMBEDDINGS[i]])
        neighbor_indices = neighbor_indices[0]

        # Get ratings from neighbors
        neighbor_ratings = USER_MOVIE_MATRIX[u, neighbor_indices].toarray()[0]
        valid_ratings = neighbor_ratings[neighbor_ratings != 0]

        knn_pred = valid_ratings.mean() if len(valid_ratings) > 0 else np.nan
    except Exception as e:
        print(f"KNN prediction failed: {e}")
        knn_pred = np.nan

    # Combine predictions
    combined_pred = (svd_pred + knn_pred) / 2 if not np.isnan(knn_pred) else svd_pred

    # Add global mean and clip
```

```
    prediction = combined_pred + GLOBAL_MEAN
    return np.clip(prediction, 0.5, 5.0)
```

Recommendation

- Embedding-Based Neighbor Discovery: Uses user embedding from the SVD component to find k most similar users in the reduced-dimensional space rather than using raw rating vectors.
- Weighted Rating Aggregation: Computes predicted ratings for all movies by taking a similarity-weighted average of neighbor users' ratings, with closer neighbors having more influence.
- Novel Item Filtering: Excludes movies the user has already rated to ensure recommendations focus only on new, unseen content.
- Top-N Selection: Ranks unrated movies by their weighted predicted scores and returns the top N highest-rated movies as personalized recommendations for each user.

```
def get_batch_recommendations(pipeline, train_data, user_ids, k=5, top_n=10):

    all_movie_ids = MOVIE_CATEGORIES  # Use your predefined categories
    user_rated_movies = train_data.groupby('userId')['movieId'].apply(set)
    recommendations = {}

    for user_id in user_ids:
        if user_id not in user_reverse_mapping:
            recommendations[user_id] = np.array([])
            continue

        user_idx = user_reverse_mapping[user_id]

        # Get user embedding instead of raw vector
        user_embedding = USER_EMBEDDINGS[user_idx].reshape(1, -1)

        # Find similar users in embedding space
        distances, neighbor_indices = HYBRID_MODEL.kneighbors(
            user_embedding,
            n_neighbors=k+1
        )
        neighbor_indices = neighbor_indices.flatten()[1:]
        similarity_weights = 1 - distances.flatten()[1:]

        # Compute weighted ratings using SVD approximation
        weighted_ratings = USER_MOVIE_MATRIX[neighbor_indices].multiply(
            similarity_weights[:, np.newaxis]
        ).sum(axis=0)
        weighted_ratings = np.array(weighted_ratings).flatten() + GLOBAL_MEAN
```

```
        # Filter out already rated movies
        rated = user_rated_movies.get(user_id, set())
        unrated_mask = [movie_id not in rated for movie_id in all_movie_ids]
        unrated_ratings = weighted_ratings[unrated_mask]
        unrated_movies = all_movie_ids[unrated_mask]

        # Get top recommendations
        top_indices = np.argsort(unrated_ratings)[-top_n:][::-1]
        recommendations[user_id] = unrated_movies[top_indices]

    return recommendations
```

V. Evaluations

1. Data statistics

- Users: 85,307
- Movies: 10,524
- Ratings: 18,187,272
- Train size: 14,549,817
- Test size: 3,637,455

2. Evaluation metrics

- RMSD (Root Mean Squared Deviation) Measures the average magnitude of prediction errors between predicted and actual ratings.

$$RMSD = \sqrt{\frac{\sum_{t=1}^{n}(\hat{y}_t - y_t)}{n}}$$

- Recall@5 Evaluates the proportion of relevant items successfully identified within the top 'k' recommendations.

$$R = \frac{T_p}{T_p - F_n}$$

3. Pure SVD

Testing conducted on a 1,000,000-entry sample from the test set.

RSMD

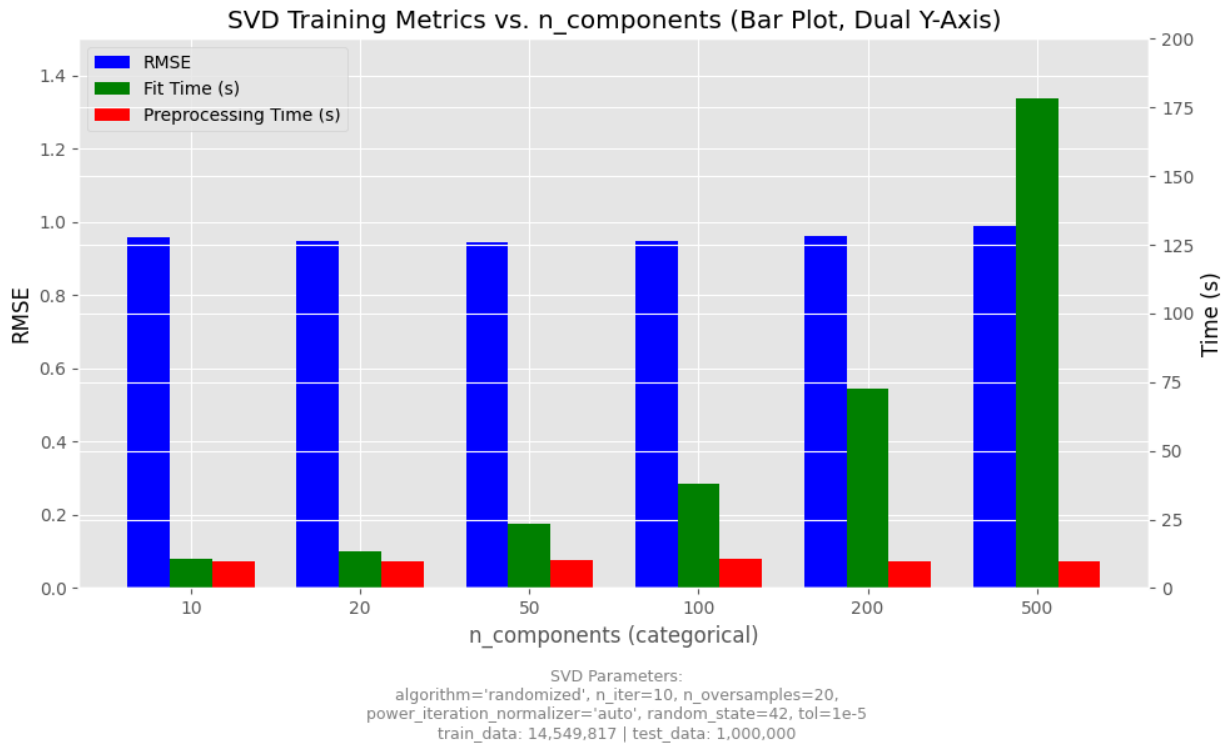

Figure 4: SVD training metrics with respect to number of components

Varying the number of latent components from 10 to 500 produced minimal variation in RMSE scores, which ranged between 0.943 and 0.988, with optimal performance achieved at 50 components.

Model fitting time increased proportionally with component count:

- 10 components: 10 seconds
- 50 components: 25 seconds
- 500 components: 180 seconds

Prediction time remained consistent at approximately 10 seconds across all configurations.

The marginal improvement in performance may not justify the substantial increase in computational requirements beyond 50 components.

Recall

Tableau 3: Recal@k scores with respect to 'k' for pure SVD recommender

'k'	**recall@**
10	0.0411
30	0.1450
50	0.2187

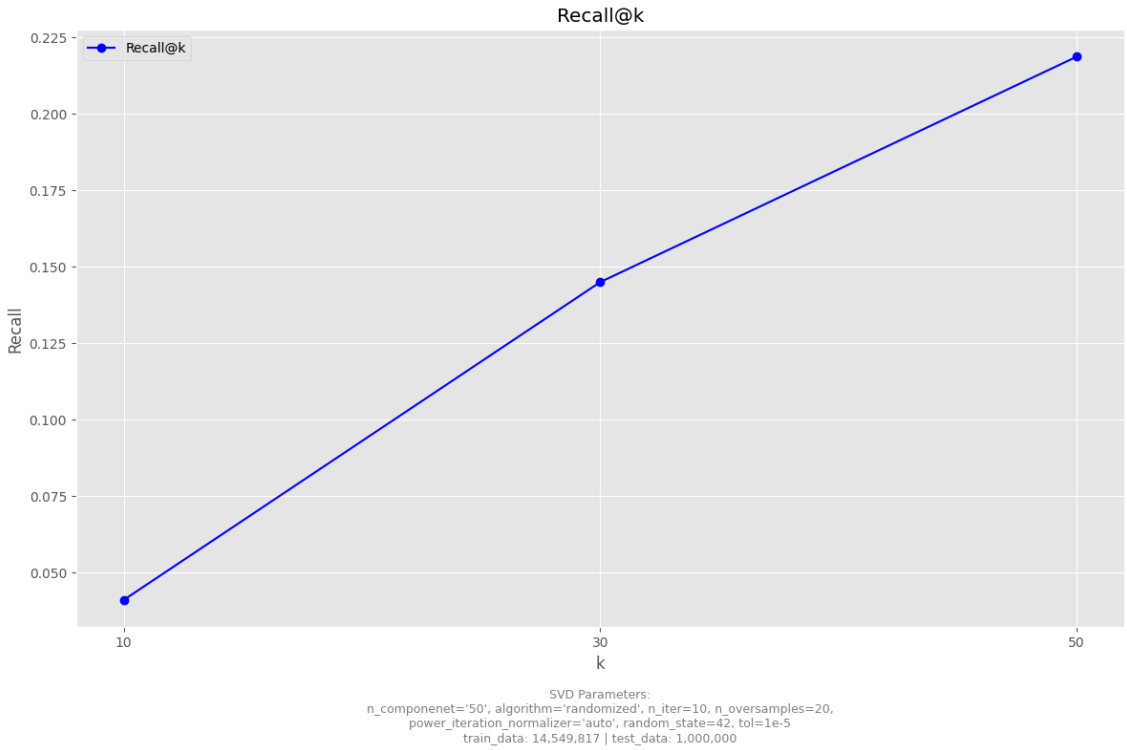

Figure 5: Recall@k score with respect to 'k' for pure SVD recommender

Initial recall scores for the top 10 recommendations were expectedly low given the dataset's scale. Even an ideal model would struggle to consistently identify the same preferred movies when the number of similarly rated options far exceeds 'k'.

Expanding recommendations to the top 50 yielded a recall score of 0.21, which falls within acceptable performance parameters for large-scale recommendation systems using simpler algorithms.

4. Nearest Neighbor

Nearest Neighbor algorithms require computing distances between a query user and all other users in the dataset. This becomes O(n) for each prediction, where n is the number of users. With millions of users and items, this creates a significant computational bottleneck.

This caused a problem computationally, given that the total size of the data contains almost 20 million records. Due to this, testing was conducted on a 1,000-entry sample due to exponential computation time requirements.

RSMD

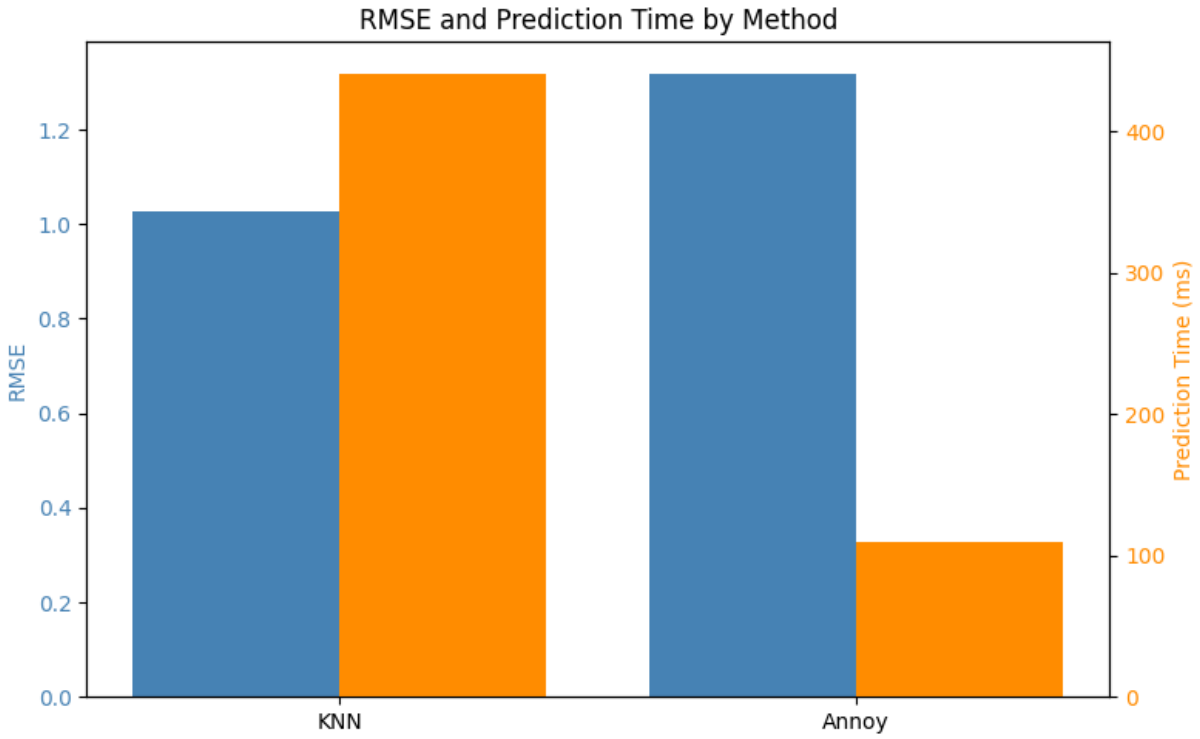

Figure 6: RMSD and prediction time comparison between KNN and Annoy models

KNN produced a RMSD score of 1.0273 while the approximation only managed 1.3194. Inversly, KNN required 5min30s in total to make the predictions, while it only took 1min50s for the second model.

These results aligned with expectations: approximation methods gain computational efficiency at the cost of recommendation precision.

Recall

Tableau 4: KNN Recall@k with respect to 'k' for pure KNN recommender

'k'	**recall@**
10	0.0912
20	0.1410
30	0.1763
40	0.2116
50	0.2302

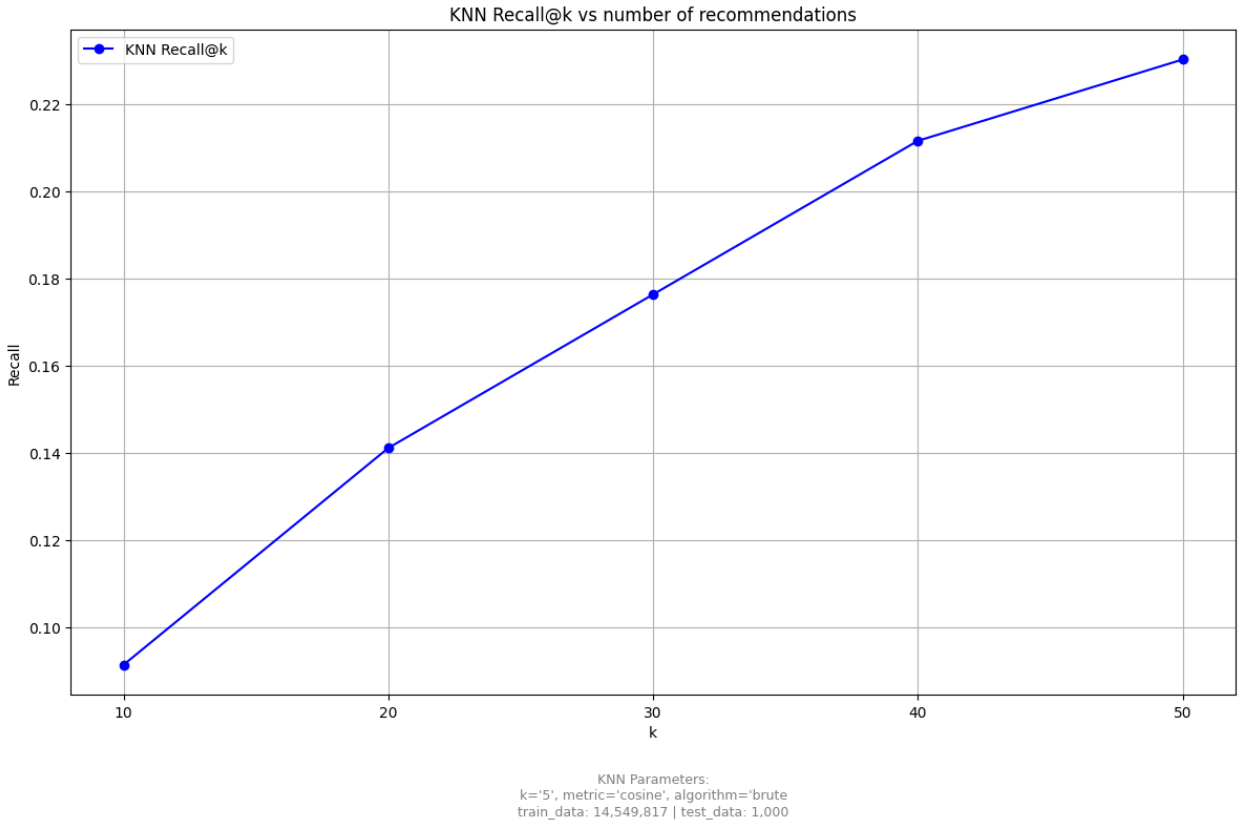

Figure 7: KNN Recall@k with respect to 'k' for pure KNN recommender

Similar to the pure SVD model, expanding recommendations yields higher recall scores of up to 0.23 at 50 recommendations. This behavior can be explained by the fact that pinpointing a small number of the most relevant options gets harder and harder as the dataset size grows. In our case, it's 'k' recommendations over a catalog of almost 90 thousand movies.

5. Hybrid

The hybrid approach introduces once more the time complexity problem of having to run each query user against all other users. However, the comparison itself shouldn't take as much, given that it will take place after the dimensionality reduction provided by the SVD method. In our case, the latent space was reduced from almost 90 thousand movies to a latent space of only 50 features.

A pure SVD approach is always the fastest between these option, but it's possible to find the balance in performance gain computation time, which might give the edge to the hybrid approach over the classic and direct methods.

RSMD

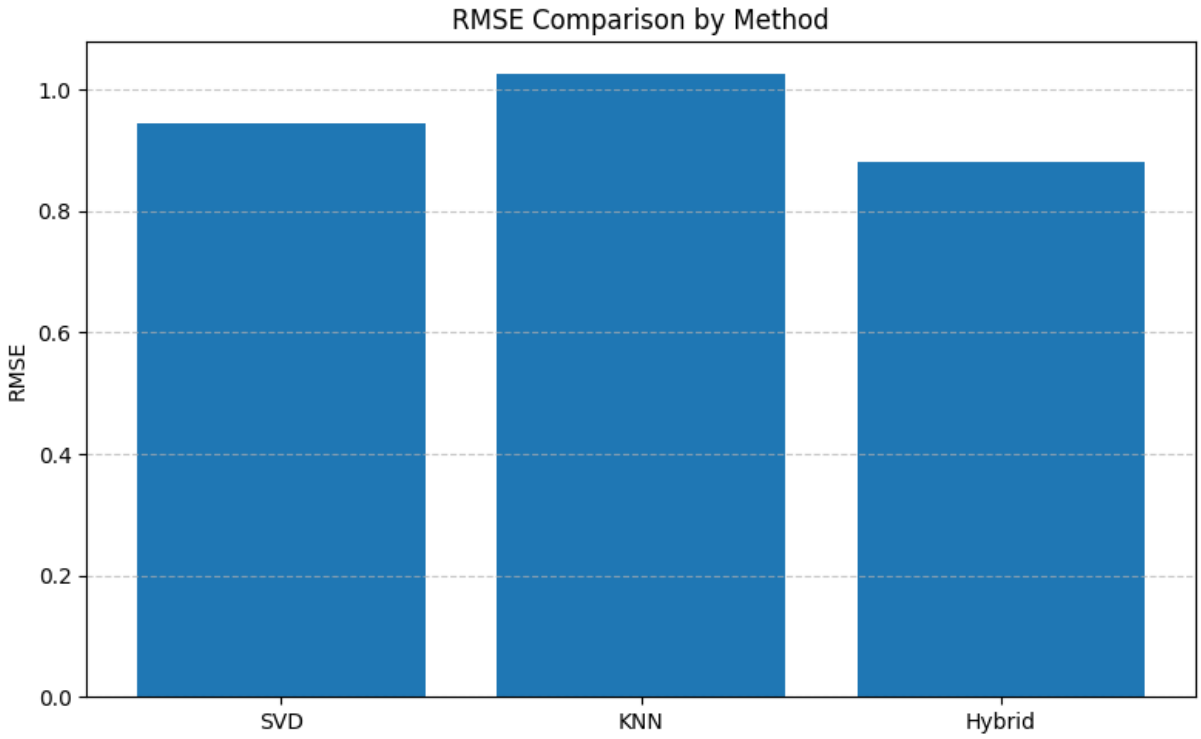

Figure 8: RMSE comparison between pure SVD, pure KNN, and a Hybrid approach

The hybrid implementation outperformed both individual models in rating prediction accuracy, with respective RMSE values of:

- SVD: 0.9437
- KNN: 1.0273
- Hybrid: 0.8821

While the hybrid approach demonstrated superior accuracy, this came with significant computational costs:

- Hybrid model: 6+ hours to process 500,000 test cases
- Pure SVD: Under 30 seconds to process 1,000,000 test cases
- Pure KNN: Computationally prohibitive for substantial test volumes

This highlights the classic accuracy-efficiency trade-off in recommendation system design.

Recall

Tableau 5: SVD-KNN Recall@k with respect to 'k'

'k'	**recall@**
10	0.0973
20	0.0867

30	0.0890
40	0.0956
50	0.1038

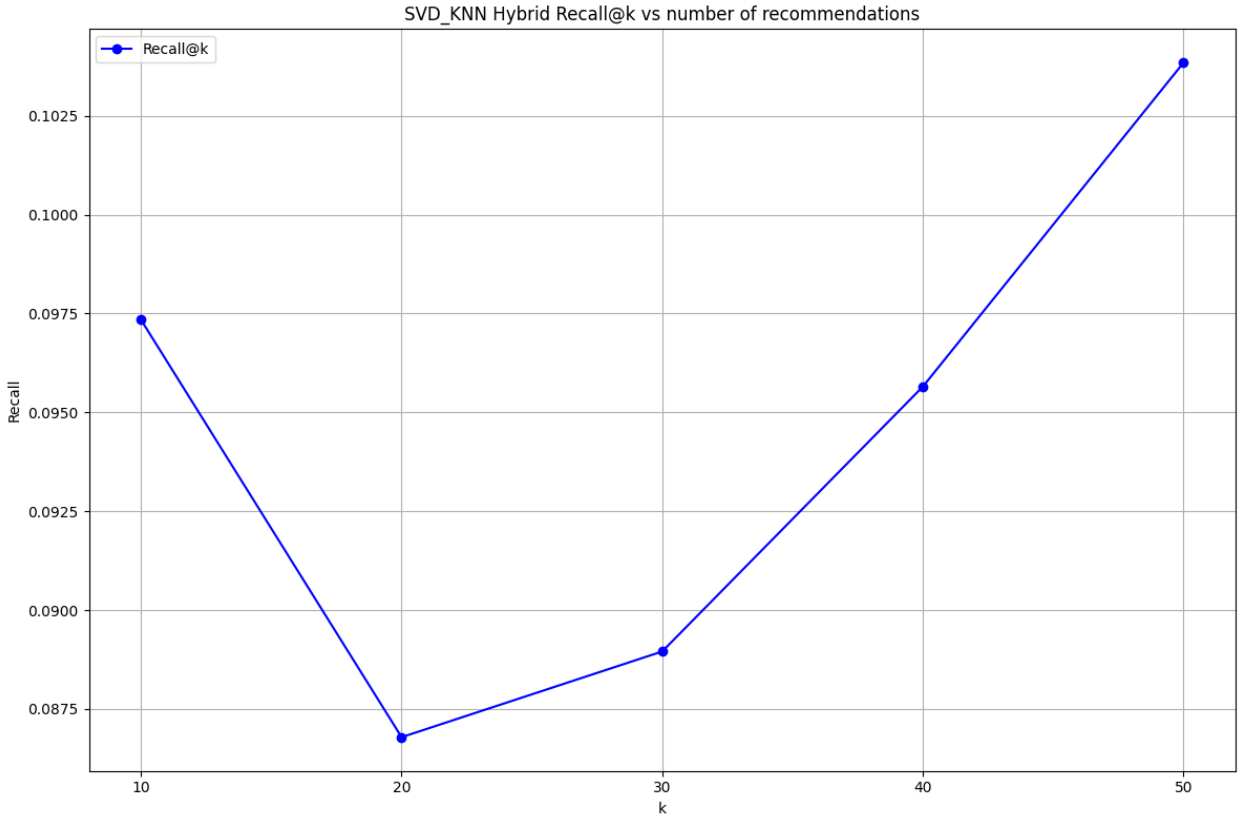

Figure 9: SVD-KNN Recall@k with respect to 'k'

Much like the previous cases, the trend of the recall score with respect to the number of recommendations is mostly ascending. This is where the similarity ends, as the hybrid method's superiority on the RMSE metric doesn't translate to the Recall score.

Conclusion

This project implemented a recommendation system using multiple collaborative filtering approaches. Our work demonstrates both the potential and limitations of various recommendation techniques.

The user-based collaborative filtering implementation using traditional K-Nearest Neighbors achieved promising accuracy with an RMSE of 1.0494 and a Recall@50 of 0.23 on test samples. However, this approach proved computationally prohibitive for large-scale applications, requiring approximately 18 hours for complete evaluation.

To address scalability concerns, we explored alternative approaches. The Approximate Nearest Neighbors (Annoy) implementation significantly improved computational efficiency, reducing prediction time by 67%, albeit with a reduction in accuracy (RMSE of 1.3194). More importantly, our Singular Value Decomposition approach and hybrid model demonstrated substantial improvements in both performance metrics and computational efficiency. The hybrid approach achieved the best accuracy with an RMSE of 0.8821, confirming our hypothesis that combining complementary methods can yield superior results.

These results highlight a fundamental consideration in recommendation system design: the balance between prediction accuracy and computational efficiency. For streaming platforms requiring real-time recommendations across millions of users and thousands of content items, optimized approaches like SVD or carefully calibrated hybrid models may prove to be the most practical solution.

Future enhancements could include:

- Implementing content-based features to mitigate the cold start problem for new users and items
- Exploring temporal dynamics to capture evolving user preferences
- Investigating deep learning approaches for potentially improved accuracy without prohibitive computational costs
- Developing personalized explanation mechanisms to increase user trust in recommendations

References

Faruk, K. O., Rahman, A., Shusmita, S. A., Awlad, M. S. I., Das, P., Mehedi, M. H. K., Iqbal, S., Rasel, A. A., Rodríguez, S., Mehmood, R., Omatu, S., Sitek, P., & Cicerone, S. (2022). K Nearest Neighbour Collaborative Filtering for Expertise Recommendation Systems. In *Distributed Computing and Artificial Intelligence, 19th International Conference* (Vol. 583, pp. 187–196). Springer International Publishing AG. https://doi.org/10.1007/978-3-031-20859-1_19

Jena, K. K., Bhoi, S. K., Mallick, C., Jena, S. R., Kumar, R., Long, H. V., & Son, N. T. K. (2022). Neural model based collaborative filtering for movie recommendation system. International Journal of Information Technology (Singapore. Online), 14(4), 2067–2077. https://doi.org/10.1007/s41870-022-00858-4

Sharma, S., Dubey, G. P., Shakya, H. K., Motwani, D., Garg, L., Brigui, I., Dewangan, B. K., Sisodia, D. S., Kesswani, N., & Shukla, R. N. (2024). Hybrid Filtering Methods in Movie Recommendation: The Enhanced SOM Approach. In AI Technologies for Information Systems and Management Science (pp. 174–187). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-70789-6_14

Skotis, A., & Livas, C. (2024). Prominent User Segments in Online Consumer Recommendation Communities: Capturing Behavioral and Linguistic Qualities with User Comment Embeddings. Information (Basel), 15(6), 356-. https://doi.org/10.3390/info15060356

Verma, R., & Diwan, A. (2023). Movie Recommendation System by Using Collaborative Filtering and Louvain Algorithm. 2023 IEEE 11th Region 10 Humanitarian Technology Conference (R10-HTC), 1009–1015. https://doi.org/10.1109/R10-HTC57504.2023.10461810

Xu, Q., & Han, J. (2021). The Construction of Movie Recommendation System Based on Python. 2021 IEEE 2nd International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA), 2, 208–212. https://doi.org/10.1109/ICIBA52610.2021.9687872

Kumar, P., Kibriya, S. G., Ajay, Y., & Ilampiray. (2021). Movie Recommender System Using Machine Learning Algorithms. Journal of Physics. Conference Series, 1916(1), 12052-. https://doi.org/10.1088/1742-6596/1916/1/012052

Sadhasivam, J., Cera, J. M., Deepa, R., Satheshkumar, K., Muthukumaran, V., Satheesh Kumar, S., & Angeline Kavitha, M. (2021). Movie recommendation system using clustering mining with Python. Journal of Physics. Conference Series, 1964(4), 42073-. https://doi.org/10.1088/1742-6596/1964/4/042073

pandas. (2024, September 20). *pandas - Python Data Analysis Library*. https://pandas.pydata.org/